

PRAC: Principal-Random Subspace for LLM Activation Compression and Memory-Efficient Training

Yanyi Li^{*1} Yimu Zhang^{*1} Cong Fang¹

Abstract

Activations have become the primary memory bottleneck in large-batch LLM training. However, existing compression methods fail to exploit the spectral structure of activations, resulting in slow convergence or limited compression. To address this, we bridge the relationship between the algorithm’s fast convergence and the requirements for subspace projection, and show that an effective compression should yield an unbiased estimate of the original activation with low variance. We propose **Principal-Random Subspace for LLM Activation Compression (PRAC)**, which novelly decomposes activations into two components: a principal subspace captured via SVD to retain dominant information, and a random subspace sampled from the orthogonal complement to approximate the tail. By introducing a precise scaling factor, we prove that PRAC yields an unbiased gradient estimator with minimum variance under certain conditions. Extensive experiments on pre-training and fine-tuning tasks demonstrate that PRAC achieves up to 36% total memory reduction with negligible performance degradation and minimal computational cost.

1. Introduction

Memory footprint has emerged as a critical bottleneck in the training of large language models (LLMs). During training, the memory overhead primarily consists of three components: model parameters (weights), optimizer states, and activations. The activations represent the intermediate outputs of each layer computed during the forward pass, which are typically retained to compute gradients in the subsequent

^{*}Equal contribution ¹State Key Lab of General AI, School of Intelligence Science and Technology, Peking University, China. Correspondence to: Cong Fang <fangcong@pku.edu.cn>.

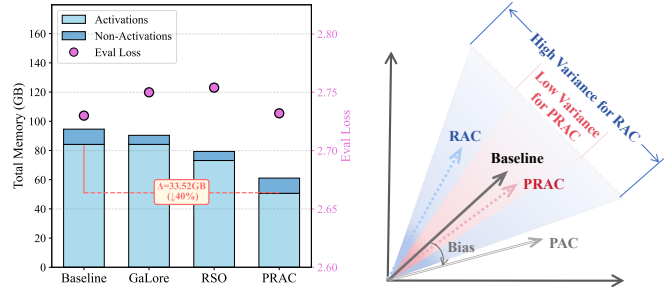


Figure 1. Left: Performance comparison on LLaMA-1B. PRAC achieves superior performance compared to existing compression methods, while reducing activation memory by 40% (resulting in a 36% reduction in total memory). **Right:** A conceptual comparison of subspace strategies. PAC results in biased reconstruction, while RAC leads to high variance in gradient estimation. In contrast, PRAC ensures unbiased estimates while achieving minimum variance under Activation Degenerate Condition.

backward propagation. In practice, a moderately large batch size generally promotes more stable training dynamics and improves GPU parallelism utilization, thereby accelerating convergence efficiently. However, a larger batch size substantially increases the activation memory footprint, turning it into the primary bottleneck and limiting both training efficiency and scalability. As illustrated in Figure 1 (Left), training a LLaMA-1B model (batch size¹ of 128 and sequence length of 256) requires 94.5GB of memory, with model parameters 2.48GB, optimizer states and gradients 7.95GB for commonly used Adam algorithm (Kingma & Ba, 2014), and activation 84.17GB, where activations account for a staggering 89% of the total usage.

A straightforward approach to reducing activation memory is to modify the implementation of the training algorithm while preserving its underlying logic. For instance, gradient checkpointing (Chen et al., 2016) selectively re-computes certain activations during the back-propagation phase, rather than storing them throughout the forward pass. This technique introduces additional computational overhead in the backward pass, thereby increasing overall training time.

¹In this paper, “batch size” denotes the micro-batch size, representing the maximum number of samples processed during a single forward and backward pass on an individual GPU.

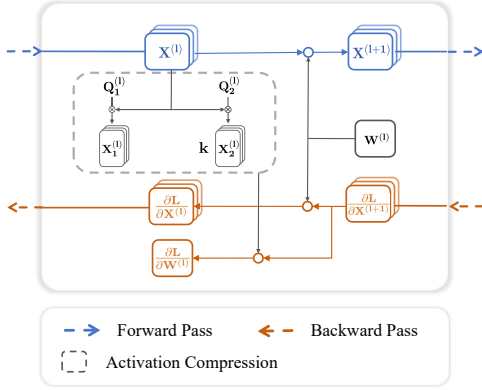


Figure 2. The flowchart of PRAC. During the forward pass (blue line), PRAC projects activations onto both the principal and random subspaces, storing the corresponding projection matrices $Q_1^{(l)}, Q_2^{(l)}$ and compressed values $X_1^{(l)}, X_2^{(l)}$. During the backward pass (orange line), the activations are reconstructed from these stored components.

The other line of research focuses on reducing activation memory through novel algorithmic design. The core idea of such approaches is to iteratively optimize within a special chosen subspace of the model parameters. However, this remains a significant challenge, especially when aiming to maintain negligible performance degradation.

One representative example is zeroth-order optimization, which has attracted considerable attention in recent years, leading to multiple proposed variants (Malladi et al., 2023; Gautam et al., 2024; Chen et al., 2024b; Zhang et al., 2024b; Zhao et al., 2024b; Shu et al., 2025; Petrov et al., 2025). From an algorithmic perspective, the algorithm per-step samples a random direction ξ , typically drawn from a standard normal or uniform spherical distribution, and constructs a gradient estimator of the form $\frac{1}{\eta}[f(w + \eta\xi) - f(w)]\xi$, where w denotes the model parameters and η is a small positive constant. Intuitively, as $\eta \rightarrow 0$, this estimator approximates $[\xi^\top \nabla f(w)]\xi$, which corresponds to performing an update only along the normalized direction $\xi/\|\xi\|$, scaled by the directional derivative of the objective function. In implementation, such methods only require access to the final output of the forward pass, allowing intermediate activations to be released immediately. This substantially reduces the peak memory footprint during training. However, since these approaches typically update only a one-dimensional subspace per iteration, they converge much more slowly than first-order methods in practice and are generally not suitable for challenging tasks such as large-scale pre-training.

To improve efficiency, some advanced methods incorporate architectural information to enable multi-dimensional subspace updates. For example, RSO (Random Subspace Optimization) introduced by Chen et al. (2025) in the outer-loop uniformly samples subspaces and in the inner-loop

optimizes weights in the subspaces by inserting a low-dimensional trainable module. Activation memory is reduced because only the projected activations need to be stored—a principle inspired by LoRA (Hu et al., 2022) and ReLoRA (Lialin et al., 2023). Despite these memory savings, this method still results in non-negligible performance degradation during pre-training, limiting its practical applicability.

We emphasize that a central limitation of existing activation-efficient methods is their inability to leverage the spectral structure of activations, which ultimately compromises training effectiveness. In this paper, we directly focus on how to compress activations during training with minimal impact on convergence rate. Following Zhao et al. (2024a); He et al. (2024); Malladi et al. (2023); Gautam et al. (2024), we still adopt a linear compression scheme: given a projection matrix $P \in \mathbb{R}^{n \times r_1}$, the activation matrix X is compressed as XP and reconstructed as $XP P^\top$.

Our starting point is a standard stochastic optimization formulation, through which we bridge the relationship between the algorithm’s fast convergence and the requirements for subspace projection. We demonstrate that a well-chosen projection should yield an unbiased estimate of the original activation with low variance that ensures provable and efficient training convergence.

To design an efficient estimator, we leverage the spectral structure of activations. We formalize widely observed “low-rank” phenomenon (Hu et al., 2022; Zhao et al., 2024a; Chen et al., 2024a; Zhang et al., 2025) as the *degenerate condition*, which assumes that the singular values of activations consist of a few dominant ones, followed by a long tail of smaller values whose cumulative energy (squared sum) is bounded.

The primary contribution of this paper is the introduction of an activation compression technique, termed PRAC (Principal-Random Subspace for LLM Activation Compression). We show this method is optimal in the sense that it achieves *minimum* variance among all unbiased estimators under the degenerate condition.

Figure 2 illustrates the overall workflow of PRAC during forward and backward propagation. PRAC consists of two simple components: a principal subspace projection, which obtains a subspace Q_1 via SVD, and a random subspace projection, where Q_2 is sampled uniformly from the orthogonal complement of Q_1 . Additionally, a *key* scaling factor k is introduced to ensure the unbiasedness of the estimator.

$$\tilde{X} = (XP)P^\top = (XQ_1)Q_1^\top + k(XQ_2)Q_2^\top.$$

In implementation, PRAC employs a dynamic subspace update schedule that refreshes projection bases at fixed intervals, thereby rendering the cost of SVD and QR decompositions negligible. The method further maximizes memory

efficiency through subspace sharing and a layer-wise policy (see details in Section 5).

We conduct extensive experiments on both pre-training and fine-tuning tasks. Our experiments demonstrate that PRAC matches or surpasses baselines across various tasks, achieving up to 36% memory saving while maintaining competitive performance.

Our key contributions are summarized as follows:

- We propose PRAC, a novel activation compression method that effectively integrates principal and random subspace projections. PRAC is the *first* method to leverage the structural information of activations for memory-efficient pre-training. Theoretically, we prove that PRAC yields an unbiased estimator with *minimum* variance under the Activation Degenerate Condition.
- We demonstrate the practical efficacy of PRAC through extensive experiments on both pre-training and fine-tuning tasks. PRAC consistently achieves substantial memory reduction with negligible performance loss and very small computational overhead.

2. Related Works

Activation-Efficient Methods. To alleviate activation memory overhead, a technique modifying the checkpointing mechanism during backpropagation was proposed in Chen et al. (2016). This approach has not only achieved widespread adoption but has also catalyzed subsequent research into modifications of the backpropagation algorithm itself (Yang et al., 2024; Luo et al., 2025).

Furthermore, alternative approaches employ zeroth-order optimization to bypass backpropagation entirely (Malladi et al., 2023; Gautam et al., 2024; Chen et al., 2024b; Miles et al., 2024; Chen et al., 2025; Shamshoum et al., 2025; Petrov et al., 2025), while others retain first-order methods but address the optimization problem within a low-dimensional subspace (Chen et al., 2025).

Compression-based approaches (Yang et al., 2024; Shamshoum et al., 2025; Miles et al., 2024) offer an alternative strategy by approximating intermediate variables. However, these methods are generally designed for specific sub-components of the model architecture. Furthermore, they fail to explicitly address how compression artifacts influence the convergence rate, limiting their potential in general-purpose activation compression for memory-efficient training.

Optimization States-Efficient Methods. Considerable research aims to improve the memory efficiency for optimizer states. GaLore (Gradient Low-Rank Projection) and its variants (Zhao et al., 2024a; Liang et al., 2024; He et al.,

2024) perform low-rank compression on gradients. To address the compression of momentum terms in the Adam optimizer, AdaFactor (Shazeer & Stern, 2018) utilizes factorization techniques to approximate the storage of the second moment. Adam-mini (Zhang et al., 2024a) designs a block-wise learning rate strategy based on the heterogeneity of the neural network’s Hessian matrix, reducing the memory footprint of the second moment. Fira (Chen et al., 2024a) and Apollo (Zhu et al., 2025) utilize optimizer states to update adaptive learning rates. However, these algorithms do not modify the backpropagation process and thus cannot reduce the memory usage of activations, which is the dominant factor. Our method is orthogonal to this kind of method and can be combined with them for additional memory savings.

3. Starting Point of Activation Compression

3.1. Problem Formulation

The training of Large Language Models (LLMs) can be formulated as the following stochastic optimization problem:

$$\min_W f(W) := \mathbb{E}_\zeta[F(W; \zeta)], \quad (1)$$

where W denotes the model parameters and ζ represents the random data batch. We simply study the Stochastic Gradient Descent (SGD) (Bottou, 2010) method, noting that the analysis extends naturally to other optimizers. The update of SGD goes as:

$$W_{t+1} = W_t - \eta_t \underbrace{\nabla F(W_t; \zeta)}_{\text{update term}}. \quad (2)$$

To mitigate memory bottlenecks, various methods compress training artifacts (parameters, gradients, or activations). This compression replaces the exact gradient with an approximate update term, denoted as $\tilde{G}(W_t; \zeta)$, leading to the modified update rule:

$$W_{t+1} = W_t - \eta_t \tilde{G}(W_t; \zeta). \quad (3)$$

A fundamental question arises: **What properties should the compression satisfy to guarantee fast convergence?**

To address this question, we first examine the impact of systematic bias in the gradient estimator.

Proposition 3.1 (Non-convergence under Constant Bias). *Consider two-dimensional strongly convex problem: $f(w^{(1)}, w^{(2)}) = \mathbb{E}_{\xi'}(w^{(1)} - 3\xi')^2 + (w^{(2)})^2$ with ξ' following a Bernoulli distribution. The parameters are initialized at $w^{(1)} = 0, w^{(2)} = 1$. At each step, only the first coordinate is updated by a stochastic gradient: $\tilde{G} = [2(w^{(1)} - 3\xi'), 0]^\top$. Then for any $\{\eta_t\}$ that satisfy the Robbins-Monro condition: $\sum_{t=0}^{\infty} \eta_t = \infty, \sum_{t=0}^{\infty} \eta_t^2 < \infty$, one has $w_t^{(2)} = 1$ for all t . Consequently, $(w_t^{(1)}, w_t^{(2)})$ will never converge to an approximate stationary point.*

In the simple example above, the gradient estimator is biased by $\Theta(1)$ because $w^{(2)}$ is never updated. It is interesting to observe that when $|w^{(1)}| < 2$, the magnitude of the stochastic gradient for $w^{(1)}$ (i.e., $2|w^{(1)} - 3\xi'|$) is always larger than that for $w^{(2)}$ (i.e., $2|w^{(2)}|$). Consequently, even if $w^{(1)}$ converges to its minimizer 0, the maximum selection rule that consistently picks the coordinate with the largest gradient magnitude will never update $w^{(2)}$. It implies that principal methods, such as GaLore (Zhao et al., 2024a) may fail to converge in general stochastic optimization settings. Proposition 3.1 also establishes that a constant biased estimator cannot ensure convergence. We then consider unbiased estimator with non-negligible variance.

Proposition 3.2 (Convergence Rate with Bounded Gradient Variance, Ghadimi & Lan (2013)). *Let the objective function f be L -smooth and bounded below, and denote $\Delta = f(W_1) - \inf f$. Assume that at each iteration t we have access to an unbiased stochastic gradient $\tilde{G}(W_t, \zeta_t)$ satisfying $\mathbb{E}[\tilde{G}(W_t, \zeta_t) | W_t] = \nabla f(W_t)$, $\mathbb{E}[\|\tilde{G}(W_t, \zeta_t) - \nabla f(W_t)\|^2 | W_t] \leq \sigma^2$. Consider the update rule in (3) with constant step sizes η_t and then randomly select an output W_R from $\{W_1, \dots, W_T\}$ according to a certain probability mass function. Then the following bound holds:*

$$\mathbb{E}[\|\nabla f(W_R)\|^2] \leq \frac{2L\Delta}{T} + 2\sqrt{\frac{2L\Delta}{T}}\sigma.$$

Proposition 3.2 is a standard result in non-convex optimization (Nesterov, 2013; Ghadimi & Lan, 2013). It demonstrates that unbiased compression ensures convergence. Moreover, the convergence rate is hindered by the variance σ^2 . Therefore, an effective compression should yield a gradient estimator that is both **unbiased** and **low-variance**.

Since activations dominate memory usage in LLM training, we translate the gradient-level requirements established above into concrete criteria for activation compression.

3.2. Criteria for Activation Compression

The theoretical constraints established in Section 3.1 for general gradients directly inform the design of activation compression. For a linear layer, such as the Query layer in the attention block, the quality of the activation reconstruction dictates the quality of the gradient estimate.

Lemma 3.3. *Consider a linear layer with forward pass $Y = XW$. Let $\tilde{X}$ be a compressed estimator of the activation X . If $\tilde{X}$ is unbiased ($\mathbb{E}[\tilde{X}] = X$) with bounded variance $\mathbb{E}[\|\tilde{X} - X\|_F^2] \leq \sigma^2$, then the resulting gradient estimator $\tilde{G} = \tilde{X}^\top (\nabla_Y L)$ satisfies:*

1. **Unbiasedness:** *The gradient estimator is unbiased with respect to the true gradient:*

$$\mathbb{E}[\tilde{G}] = \mathbb{E}[\tilde{X}^\top](\nabla_Y L) = \nabla_W L.$$

2. **Bounded Variance:** *The gradient variance is bounded by the activation variance and the upstream gradient norm:*

$$\mathbb{E}[\|\tilde{G} - \nabla_W L\|_F^2] \leq \sigma^2 \|\nabla_Y L\|_2^2.$$

Lemma 3.3 shows that constructing an unbiased and low-variance approximation of activations suffices to satisfy the convergence guarantees in Propositions 3.1 and 3.2.

Following Zhao et al. (2024a); He et al. (2024); Malladi et al. (2023); Gautam et al. (2024), we adopt a subspace projection approach. Let $P \in \mathbb{R}^{n \times r}$ be a projection matrix, and the activation X is compressed to a lower-dimensional representation XP and reconstructed as $\tilde{X} = (XP)P^\top$. The central challenge, therefore, lies in designing the projection matrix P such that the reconstruction $\tilde{X}$ minimizes variance while maintaining unbiasedness, specifically tailored to the spectral structure of the activations.

4. Proposed Method: PRAC

In this section, we introduce PRAC, a hybrid framework motivated by the spectral structure of activations. By integrating Principal and Random Subspace to address their respective limitations, we balance the bias-variance trade-off. Finally, we prove that PRAC yields the minimum variance unbiased estimator under the degenerate condition.

4.1. Spectral Analysis of Activations

To design an effective projection matrix P in Section 3.2, we analyze the spectral structure of the activations using the 10th layer of LLaMA-130M (at 10% training progress) as an example. As shown in Figure 3, the spectrum exhibits two distinct characteristics: a few dominant singular values capture the majority of the spectral energy; and the remaining singular values form a long tail that decays slowly.

Note that the “low-rank” structure is commonly observed in weight matrices (Hu et al., 2022; Lialin et al., 2023). We study the activation matrix and emphasize that the long-tail directions cannot be ignored—truncating them significantly slows down convergence. We introduce the term *degenerate condition* to characterize this observed phenomenon.

Assumption 4.1 (Activation Degenerate Condition). For activation matrix X , let σ_i be the i -th largest singular value of the activations. We say X satisfies the (s, q) -degenerate condition ($(s, q) - D$ in short) if it satisfies: $\sum_{s+1}^n \sigma_i^2 \leq q$.

This assumption posits that only the top s singular values of the activation matrix are large. Since the remaining singular values are typically small and roughly equal in practice, we bound the sum of squares of the remaining singular values (same as the square of the Frobenius norm for the rest matrix) by a constant q .

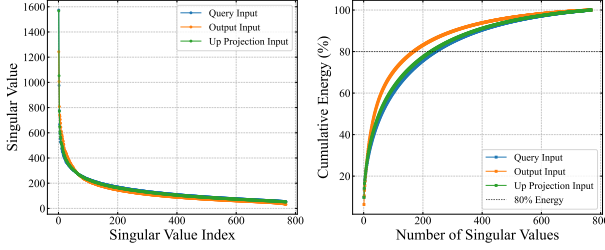


Figure 3. Activation Degenerate Condition: Singular value spectrum (Left) and cumulative energy ratio (Right) of activations from the 10th layer of LLaMA-130M (at 10% training progress), where certain activations dominate while the remainder decay slowly, accounting for under 20% of the total energy.

Based on Assumption 4.1, we can conduct a theoretical analysis of the optimal design of the projection matrix P (see Section 4.3). Before that, let us first introduce two key components of PRAC, which are also subspace projection methods.

4.2. Key Components of PRAC

Intuitively, one might consider two natural approaches to activation compression: projecting onto the principal subspace, or onto a random subspace. However, both have inherent drawbacks that our method addresses.

Principal Subspace for Activation Compression (PAC). PAC aims to retain the most significant information by projecting activations onto their principal subspaces. Let the Singular Value Decomposition (SVD) of the activation matrix be $X = U\Sigma V^T = \sum_{i=1}^n s_i u_i v_i^T$. We define the projection matrix $Q_1 = [v_1^{(1)}, \dots, v_{r_1}^{(1)}] \in \mathbb{R}^{n \times r_1}$ using the top- r_1 right singular vectors. Then XQ_1 captures the rank- r_1 principal information. For the rank $r_1 < n$, the method introduces a systematic reconstruction bias $\Delta = \|X - XQ_1Q_1^T\|_F^2 = \sum_{i=r_1+1}^n \sigma_i^2 > 0$. As discussed in Section 3.2, this bias can lead to non-convergence in the optimization problem (1).

Random Subspace for Activation Compression (RAC). To avoid bias, RAC projects activations onto a random subspace sampled uniformly from the Stiefel manifold St_{n,r_2} , defined as $\text{St}_{n,r} = \{P \in \mathbb{R}^{n \times r} \mid P^T P = I_r\}$. The construction proceeds as follows: First, we generate a random matrix $S \in \mathbb{R}^{n \times r_2}$ with entries $S_{ij} \sim \mathcal{N}(0, 1)$. We then perform QR decomposition on S to obtain an orthogonal basis $Q_2 \in \mathbb{R}^{n \times r_2}$, such that $Q_2^T Q_2 = I$.

While RAC can provide an unbiased estimator of the activation (via setting a scaling factor), the inherent randomness introduces high variance: $\mathbb{E}\|\tilde{X}_{\text{rac}} - X\|_F^2 \leq (\frac{n}{r_2} - 1) (\sum_{i=1}^s \sigma_i^2 + q)$ due to its top singular values. (see Theorem C.7 and C.9 in Appendix C.2). This high variance destabilizes the training process and slows convergence.

To sum up, PAC is data-adaptive but biased, while RAC is unbiased but suffers from high variance precisely because it is not adaptive to activation statistics. PRAC bridges this gap by combining an adaptive principal subspace derived from activation statistics with a random subspace from its orthogonal complement, achieving both unbiasedness and low variance.

4.3. Optimal Design via Hybrid Projection

We first establish a lower bound on the variance to estimate the activations. Specifically, for any activation X , we consider the random subspace projection method where the projection matrix $P \in \mathbb{R}^{n \times r}$ is drawn from a distribution $\mathbb{P}_X$, which must satisfy the unbiasedness condition (i.e. $\mathbb{E}_{P \sim \mathbb{P}_X} [XPP^T] = X$) and depend on X , then the lower bound variance can be written as

$$\sup_{X \sim (s, q) - D} \left(\inf_{\substack{P \sim \mathbb{P}_X, \\ \mathbb{E}_{P \sim \mathbb{P}_X} [XPP^T] = X}} \mathbb{E}_{\mathbb{P}_X} \|XPP^T - X\|_F^2 \right). \quad (4)$$

For this min-max optimization problem, if the small singular values of X exhibit non-uniformity, P can be tailored to exploit this structure. Intuitively, a distribution where X possesses approximately uniform tail singular values represents a relatively worst-case scenario. The following lemma gives the lower bound explicitly.

Lemma 4.2 (Lower Bound). *Under Assumption 4.1, when $s < r < n$, we have (4) $\geq (\frac{n-s}{r-s} - 1)q$.*

We now propose our PRAC and demonstrate that its variance achieves the lower bound in Lemma 4.2, thereby demonstrating its optimality in attaining the minimum variance.

Construction of the Projection Matrix. We first extract the principal basis vectors $v_i^{(1)} (i \in [1, r_1])$ via SVD. Then, sampling a random matrix from the complementary orthogonal subspace:

$$S_o = (I - Q_1 Q_1^T) S, \quad \text{where } S_{ij} \stackrel{i.i.d.}{\sim} \mathcal{N}(0, 1). \quad (5)$$

Let $v_j^{(2)} (j \in [1, r_2])$ be the column vectors obtained from $\text{QR}(S_o)$. By construction, the principal and random subspaces are orthogonal ($v_i^{(1)} \perp v_j^{(2)}$). The unified projection matrix $P \in \mathbb{R}^{n \times (r_1 + r_2)}$ is defined as:

$$P = \left[\underbrace{v_1^{(1)}, \dots, v_{r_1}^{(1)}}_{\text{Principal}}, \underbrace{\sqrt{k}v_1^{(2)}, \dots, \sqrt{k}v_{r_2}^{(2)}}_{\text{Random}} \right], \quad (6)$$

where k is a scaling factor used to ensure unbiased reconstruction. It is critical that, since we only sample a subspace of dimension r_2 to represent the entire tail of dimension

Table 1. Comparison of Estimation Unbiasedness and Variance across Subspace Projection Methods.

Algorithm	Form	Scaling	Unbiasedness	Variance
PAC	$\tilde{X}_{\text{pac}} = (XQ_1)Q_1^\top$	-	×	-
RAC	$\tilde{X}_{\text{rac}} = (XQ_2)Q_2^\top$	n/r_2	✓	High
PRAC	$\tilde{X}_{\text{prac}} = (XP)P^\top$	$(n - r_1)/r_2$	✓	Minimum

$n - r_1$, the energy of the random component must be amplified to ensure the estimator remains unbiased in expectation.

Unbiased Estimation. By selecting an appropriate k , PRAC ensures the unbiasedness.

Theorem 4.3 (Unbiased Reconstruction of PRAC). *Let P be the projection matrix defined in (6). If the scaling factor is set to $k = \frac{n-r_1}{r_2}$, the reconstruction $\tilde{X} = XPP^\top$ satisfies $\mathbb{E}[\tilde{X}] = X$.*

Variance Reduction Analysis. A key advantage of PRAC is its ability to achieve the minimum variance.

Theorem 4.4 (Variance Bound of PRAC). *For X admitting (s, q) -D condition with $s < r < n$, by choosing $r_1 = s$ and $r_2 = r - r_1$, we have: $\mathbb{E}[\|XPP^\top - X\|_F^2] \leq (\frac{n-r_1}{r_2} - 1)q$.*

Theorem 4.3 confirms that setting $k = \frac{n-r_1}{r_2}$ yields an unbiased reconstruction of activations and, by extension, gradients (Lemma 3.3). Theorem 4.4 demonstrates its optimality in the sense that it achieves the minimum variance among all unbiased estimators under the degenerate condition (Lemma 4.2). Ablation study in Section 6.3 empirically validates this theoretical finding. A comparison of the three projection methods is presented in Table 1.

5. PRAC for Memory-Efficient Training

In this section, we detail the practical implementation of PRAC for memory-efficient LLM training. To minimize computational overhead, we introduce dynamic update strategies and subspace sharing. We further propose a layer-wise adaptation scheme that accounts for the varying sensitivity of different model components. Finally, we provide a complexity analysis demonstrating the memory efficiency of our approach.

5.1. PRAC for LLM Training

Dynamic Subspace Update. Recall that the PRAC reconstruction can be written as:

$$\begin{aligned} XPP^\top &= X \left[\sum_{i=1}^{r_1} v_i^{(1)} (v_i^{(1)})^\top + k \sum_{i=1}^{r_2} v_i^{(2)} (v_i^{(2)})^\top \right] \\ &= XQ_1Q_1^\top + kXQ_2Q_2^\top, \end{aligned} \quad (7)$$

Algorithm 1 Dynamic Activation Compression via PRAC

Require: Input activation X , ranks (r_1, r_2) , update intervals (T_1, T_2) , current step t

Ensure: Projection matrices $Q_1^{(t)}, Q_2^{(t)}$, Compressed Activations $X_1^{(t)}, X_2^{(t)}$

```

1:  $k \leftarrow (n - r_1)/r_2$ 
2: // Update Principal Subspace:
3: if  $t \bmod T_1 = 0$  then
4:    $Q_1^{(t)} \leftarrow$  top- $r_1$  right singular vectors of  $X$ 
5: else
6:    $Q_1^{(t)} \leftarrow Q_1^{(t-1)}$ 
7: end if
8: // Update Random Subspace
9: if  $t \bmod T_2 = 0$  then
10:  Sample  $S \sim \mathcal{N}(0, I_{n \times r_2})$ 
11:   $Q_2^{(t)} \leftarrow$  orthonormal basis of  $(I - Q_1^{(t)}Q_1^{(t)\top})S$ 
12: else
13:   $Q_2^{(t)} \leftarrow Q_2^{(t-1)}$ 
14: end if
15: // Compression
16:  $X_1^{(t)} \leftarrow XQ_1^{(t)}, X_2^{(t)} \leftarrow k \cdot XQ_2^{(t)}$ 
17: Return:  $Q_1^{(t)}, Q_2^{(t)}, X_1^{(t)}, X_2^{(t)}$ 
    
```

where $Q_1 = [v_1^{(1)}, \dots, v_{r_1}^{(1)}], Q_2 = [v_1^{(2)}, \dots, v_{r_2}^{(2)}]$ are the principal and random projection matrices respectively.

Computing the exact principal subspace (via SVD) and generating orthogonal random projections (via QR) at every step incurs significant computational cost. To mitigate this, we employ a periodic update strategy. The principal components Q_1 and the random basis Q_2 are updated only at fixed intervals T_1 and T_2 , respectively. The rationale behind this design is that the activation statistics (and thus the related subspace) change slowly over training steps, especially after the initial warm-up phase. This lazy update mechanism drastically reduces the amortized computational overhead. The full procedure is outlined in Algorithm 1.

Subspace Sharing. In Transformer architectures, many layers often share the same input activation X such as the Query, Key and Value projections. To cut memory usage, we enforce subspace sharing: a single set of projection matrices (Q_1, Q_2) and compressed activations (XQ_1, XQ_2) is computed and shared across these layers. Beyond memory savings, this strategy ensures that gradient updates for parallel heads reside in a consistent subspace, reducing gradient noise variance and stabilizing optimization.

5.2. Layer-Wise Design

Building on the periodic update strategy, we further tailor the compression to the specific roles of different layer types, as not all layers contribute equally to training dynamics.

We adopt a layer-wise configuration tailored to the specific characteristics of linear and non-linear modules:

- **Linear Layers (MLP, Attention Projections):** Due to their large parameter scale, the updates of their weight matrices have a higher demand for gradient quality. Therefore, more activation information needs to be retained to ensure optimization effectiveness. During compression, we allocate a relatively higher rank to these layers.
- **Non-linear Layers (LayerNorm, RMSNorm, GeLU, SiLU):** These layers typically involve element-wise operations or vector-wise scaling parameters, resulting in lower information density. Although our theoretical analysis does not directly cover all cases, we find that our method remains effective for most layers. It is worth noting that certain intermediate variables in these layers, such as the mean and variance in LayerNorm, are not compressed due to their negligible memory footprint (e.g., bs vs. bsn). Likewise, operations like Flash Attention are excluded from PRAC compression because of their intricately coupled structure and engineering implementation considerations. In practice, we process all the selected non-linear layers using a lower rank.

5.3. Complexity Analysis

Theoretical Memory Footprint. We analyze memory usage in the GeLU layer ($A_u = \text{GeLU}(A'_u)$) and linear layer ($A'_d = A_u W_{\text{down}}^\top$). Standard backpropagation requires storing full tensors $A_u, A'_u \in \mathbb{R}^{b \cdot s \times n}$, costing $2bsn$. In contrast, PRAC projects these activations onto principal (r_1) and random (r_2) subspaces, caching only the low-dimensional projections in $\mathbb{R}^{b \cdot s \times (r_1 + r_2)}$. In particular, since the projection matrices are independent of the batch size, their memory overhead is negligible. The memory usage is summarized in Table 2.

Table 2. Activation memory footprint comparison for the GELU and linear layer.

	Baseline	PRAC ($r_1 + r_2 \leq \lfloor 0.6n \rfloor$)
GeLU Input	bsn	$bs(r_1 + r_2)$
W_{down} Input	bsn	$bs(r_1 + r_2)$
Total	$2bsn$	$2bs(r_1 + r_2)$

In the experiments, $r_1 + r_2$ is typically set to less than $\lfloor 0.6n \rfloor$, which results in over 40% reduction in the activations. The complete analysis for the full GPT architecture is provided in Appendix B.

Computational Overhead. The primary computational cost of PRAC arises from SVD and QR decompositions.

However, by setting the update intervals T_1 and T_2 to sufficiently large values (e.g., $T_1 = T_2 = 500$ steps), the amortized cost becomes negligible. While the projection operation XP introduces additional matrix multiplications, this overhead is minimal compared to the memory throughput gains. The experimental results of training efficiency are shown in Section 6.1.

6. Experiments

In this section, we extensively evaluate PRAC on both pre-training and fine-tuning tasks across diverse model architectures. PRAC consistently achieves up to 36% memory reduction while maintaining competitive performance.

6.1. Memory-Efficient Pre-training

Setup. We evaluate PRAC by pre-training LLaMA (Touvron et al., 2023) models (130M, 350M, 1B parameters) on the C4 dataset (Raffel et al., 2020) and GPT-2 (Brown et al., 2020) models (124M, 355M) on OpenWebText, following the standard configurations in the nanoGPT codebase². For PRAC, we configure the subspace ranks as $r_1 = r_2 = \lfloor 0.3n \rfloor$ for linear layers and $r_1 = r_2 = \lfloor 0.2n \rfloor$ for non-linear layers (e.g., RMSNorm, SiLU), where n is the hidden dimension. We compare against two memory-efficient baselines: GaLore (Zhao et al., 2024a) and RSO (Chen et al., 2025), using their reported hyperparameters. Detailed configurations are provided in Appendix D.1. To ensure statistical robustness, all experiments are conducted over multiple runs with different random seeds, and we report the averaged results.

Results. Table 3 summarizes the quantitative results. PRAC achieves the optimal trade-off between training performance and memory usage, achieving a 36% total memory reduction for the LLaMA-1B model. Figure 4 illustrates the validation loss trajectories. PRAC maintains competitive convergence throughout the entire training process. In contrast, GaLore (Principal-only) suffers from stagnation in the later stages due to bias, while RSO (Random-only) exhibits slower initial convergence due to high variance. In the GPT-2 experiments, we restrict the batch size to avoid OOM failures in the comparison methods; under these conditions, our approach achieves 20× total memory reduction.

Training Efficiency. To verify the practical speedup of PRAC, we measure training time and peak memory on LLaMA-1B using 4×A800 GPUs. As shown in Table 5, PRAC reduces peak memory by approximately 27% at a fixed batch size of 64 (to prevent OOM for the baseline). Crucially, this memory headroom allows for a larger batch size (increasing from 64 to 96), which reduces the total training time by 25%.

²<https://github.com/karpathy/nanoGPT>

Table 3. Comparison with memory-efficient algorithms on pretraining tasks. Validation perplexity (PPL, lower is better) and peak memory consumption (MEM, in GB, lower is better) are reported. “OOM” indicates that the run exceeded the memory capacity of a single A800 GPU (80 GB). For methods marked OOM, perplexity is measured using half the batch size specified above. Δ denotes the total memory reduction compared to the baseline model. Here, B denotes micro-batch size and S denotes sequence length.

	LLaMA-130M			LLaMA-350M			LLaMA-1B			GPT-2-124M			GPT-2-355M		
Method	PPL	MEM	Δ	PPL	MEM	Δ	PPL	MEM	Δ	PPL	MEM	Δ	PPL	MEM	Δ
Baseline	24.41	21.26	-	18.77	46.10	-	15.33	OOM	-	19.83	62.27	-	16.26	55.94	-
GaLore	25.12	21.08	-1%	19.65	45.37	-2%	15.64	OOM	-	21.01	62.15	-0%	17.88	55.64	-1%
RSO	25.41	19.57	-8%	19.57	40.36	-13%	15.72	79.32	-16%	22.31	54.79	-12%	18.29	46.90	-16%
PRAC	24.82	15.65	-27%	18.92	32.21	-30%	15.41	60.48	-36%	19.95	49.68	-20%	16.35	44.21	-21%
B / S	128/256			128/256			128/256			64/1024			32/1024		

Table 4. Comparison with memory-efficient algorithms on fine-tuning RoBERTa models. Average scores across the GLUE benchmark and the peak memory usage of activations in MRPC task are provided.

Method	Act. MEM (GB)	COLA	STS	MRPC	RTE	SST2	MNLI	QNLI	QQP	AVG
Full Fine-Tuning	0.42	62.24	90.92	91.30	79.42	94.57	87.18	92.33	92.28	86.28
RSO (rank=4)	0.29	62.47	90.62	92.25	78.70	94.84	86.67	92.29	90.94	86.10
PRAC (rank=4)	0.26 ($\downarrow$ 38%)	63.56	90.83	93.28	78.71	94.72	86.60	92.35	90.95	86.38

Table 5. Training Time Comparison of PRAC and Baseline on $4 \times$ A800 GPU for LLaMA-1B Models.

Method	Batch Size	PPL	MEM (GB)	Training Time (h)
Baseline	64	15.34	50.35	156
PRAC	64	15.42	36.90	179
	96 ($\uparrow$ 50%)	15.39	48.70	117 ($\downarrow$ 25%)

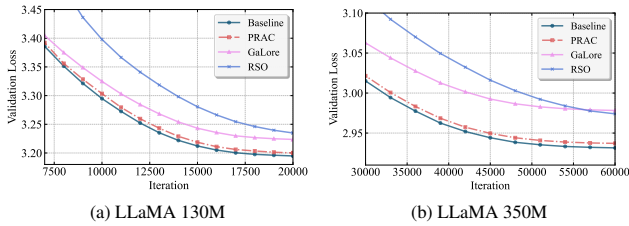


Figure 4. Loss curves of pre-training LLaMA-130M and LLaMA-350M model.

Scaling Laws. We validate the scalability of PRAC by training a LLaMA series ranging from 35M to 1B parameters. Following Chinchilla scaling laws (Hoffmann et al., 2022), we set the token budget to $20 \times$ the parameter count. Figure 5(a) shows that PRAC tracks the AdamW baseline loss curves almost identically across all scales. This near-perfect alignment is achieved while reducing activation memory by approximately 30% across all model sizes. The linear fit in Figure 5(b) confirms that PRAC adheres to the power-law scaling relationship, suggesting robust performance for even larger models.

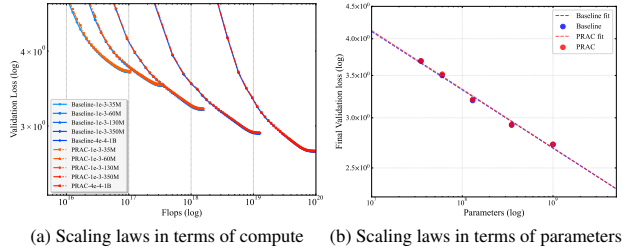


Figure 5. Scaling behavior of PRAC across model sizes (35M to 1B parameters). (a) Loss as a function of compute (FLOPs) for PRAC and AdamW baseline. PRAC maintains near-identical loss curves while using around 30% less memory. (b) Dotted lines indicate linear fits of final validation loss and parameters.

6.2. Memory-Efficient Fine-tuning

We evaluate PRAC on the GLUE benchmark (Wang et al., 2018) by fine-tuning RoBERTa (Liu et al., 2019). For a fair comparison with RSO (rank $r = 4$), we set $r_1 = r_2 = 2$ for PRAC. As shown in Table 4, PRAC not only outperforms the RSO baseline but, in certain tasks, surpasses full fine-tuning. This suggests that the stochastic regularization introduced by the random subspace component may act as a beneficial regularizer during fine-tuning.

6.3. Ablation Study

We assess the efficacy of the hybrid projection by comparing PAC, RAC, and PRAC with equivalent total rank (using $r_1 + r_2$ for PRAC). As shown in Figure 6, PAC converges slowly in the later stages, where anisotropic gradient noise

dominates the true gradient (Proposition 3.1). RAC also lags due to its inability to capture principal subspace properties. PRAC overcomes these limitations, achieving faster convergence throughout the entire training process and validating the advantage of combining dual subspaces.

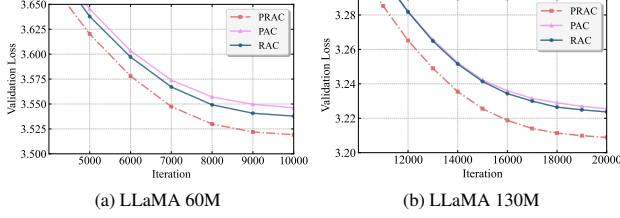


Figure 6. Loss curves of using PRAC, PAC, RAC in LLaMA series pre-training. All three methods use a total compression rank of $r = \lfloor 0.6n \rfloor$.

We evaluate the final validation loss of Adam and PRAC on LLaMA-130M under different learning rate settings after 20K steps. As shown in Figure 7 (Left), the optimal learning rates for the two optimizers are not the same. When the learning rate is set to 2.5×10^{-3} or higher, Adam exhibits a greater performance drop, whereas PRAC degrades considerably more gracefully and has a more robust range of learning rate settings.

In Figure 7 (Right), we demonstrate the performance when applying PRAC to linear layers in LLaMA-130M, under various update intervals and rank ratios. PRAC is relatively robust to changes in the update interval, which allows us to set a larger interval to avoid the time overhead incurred by operations such as SVD. Furthermore, performance remains stable above $r_1 = r_2 = \lfloor 0.3n \rfloor$ for linear layers, where n denotes the hidden dimension.

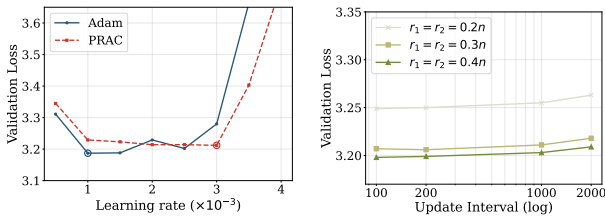


Figure 7. Left: Final validation loss on LLaMA-130M with different learning rates for Adam and PRAC. Right: Final validation loss on LLaMA-130M for PRAC with different ranks, as a function of the update interval T .

Additional results are deferred to Appendix D.3. Table 9 shows that the optimal trade-off between principal and random ranks occurs at $r_1 = r_2$, and Figure 8 confirms that the theoretically derived scaling factor $k = \frac{n-r_1}{r_2}$ proves effective for both linear and non-linear layers.

Moreover, PRAC is orthogonal to advanced optimizers such

as Muon and Adam-mini, delivering an additional approximately 30% memory reduction with negligible performance loss, as demonstrated in Figure 9 and Table 10.

Beyond algorithmic compression, common engineering techniques like Gradient Checkpointing (GC) and Gradient Accumulation (GA) also reduce memory. A detailed comparison is provided in Appendix D.3, which demonstrates PRAC’s practical advantage over these heuristic methods.

7. Conclusion

We propose PRAC, a theoretically optimal activation compression strategy that leverages the low-rank structure of LLM activations. By decomposing activations into principal and random subspaces with a tailored scaling factor, PRAC yields an unbiased gradient estimator with minimum variance under the Activation Degenerate Condition. Unlike prior subspace methods that suffer from bias or high variance, PRAC ensures stable convergence while reducing total memory by up to 36% across pre-training and fine-tuning tasks. With negligible computational overhead, PRAC offers a practical solution for training LLMs on memory-constrained hardware.

Limitations and Future Directions. Although PRAC achieves strong empirical performance and theoretical optimality for linear activation compression under the degenerate condition, two limitations remain. First, nonlinear or high-dimensional compression schemes remain under-explored and warrant further investigation. Second, due to engineering constraints, certain components (e.g., Flash Attention) are not compressed, calling for future design to integrate compression into such optimized modules.

Impact Statement

This paper presents a method to substantially improve the memory and computational efficiency of LLM training. By compressing activations via principal and random subspaces, our approach reduces the hardware requirements for pre-training and fine-tuning, thereby minimizing the energy consumption and carbon emissions associated with large-scale deep learning workloads.

Acknowledgements

This work was supported by the National Science and Technology Major Project (No. 2022ZD0114902) and the NSF China (No.s 62376008). This work was also supported in part by the Beijing Major Science and Technology Project under Contract no. Z251100008125007.

References

- Bottou, L. Large-scale machine learning with stochastic gradient descent. In Lechevallier, Y. and Saporta, G. (eds.), *Proceedings of COMPSTAT'2010*, pp. 177–186, Heidelberg, 2010. Physica-Verlag HD. ISBN 978-3-7908-2604-3.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901. Curran Associates, Inc., 2020.
- Chen, T., Xu, B., Zhang, C., and Guestrin, C. Training deep nets with sublinear memory cost. *CoRR*, abs/1604.06174, 2016.
- Chen, X., Feng, K., Li, C., Lai, X., Yue, X., Yuan, Y., and Wang, G. Fira: Can we achieve full-rank training of llms under low-rank constraint? *arXiv preprint arXiv:2410.01623*, 2024a.
- Chen, Y., Zhang, Y., Cao, L., Yuan, K., and Wen, Z. Enhancing zeroth-order fine-tuning for language models with low-rank structures. *arXiv preprint arXiv:2410.07698*, 2024b.
- Chen, Y., Zhang, Y., Liu, Y., Yuan, K., and Wen, Z. A memory efficient randomized subspace optimization method for training large language models. *arXiv preprint arXiv:2502.07222*, 2025.
- Gautam, T., Park, Y., Zhou, H., Raman, P., and Ha, W. Variance-reduced zeroth-order methods for fine-tuning language models. *arXiv preprint arXiv:2404.08080*, 2024.
- Ghadimi, S. and Lan, G. Stochastic first-and zeroth-order methods for nonconvex stochastic programming. *SIAM journal on optimization*, 23(4):2341–2368, 2013.
- He, Y., Li, P., Hu, Y., Chen, C., and Yuan, K. Subspace optimization for large language models with convergence guarantees. *arXiv preprint arXiv:2410.11289*, 2024.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D. d. L., Hendricks, L. A., Welbl, J., Clark, A., et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Lialin, V., Shivagunde, N., Muckatira, S., and Rumshisky, A. Relora: High-rank training through low-rank updates. *arXiv preprint arXiv:2307.05695*, 2023.
- Liang, K., Liu, B., Chen, L., and Liu, Q. Memory-efficient llm training with online subspace descent. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 64412–64432. Curran Associates, Inc., 2024. doi: 10.52202/079017-2054.
- Liu, J., Su, J., Yao, X., Jiang, Z., Lai, G., Du, Y., Qin, Y., Xu, W., Lu, E., Yan, J., et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- Luo, Q., Li, M., Zhao, L., and Li, X. Streambp: Memory-efficient exact backpropagation for long sequence training of llms. *arXiv preprint arXiv:2506.03077*, 2025.
- Malladi, S., Gao, T., Nichani, E., Damian, A., Lee, J. D., Chen, D., and Arora, S. Fine-tuning language models with just forward passes. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 53038–53075. Curran Associates, Inc., 2023.
- Miles, R., Reddy, P., Elezi, I., and Deng, J. Velora: Memory efficient training using rank-1 sub-token projections. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 42292–42310. Curran Associates, Inc., 2024. doi: 10.52202/079017-1338.
- Nesterov, Y. *Introductory lectures on convex optimization: A basic course*, volume 87. Springer Science & Business Media, 2013.
- Petrov, E., Evseev, G., Antonov, A., Veprikov, A., Bushkov, N., Moiseev, S., and Beznosikov, A. Leveraging coordinate momentum in signsgd and muon: Memory-optimized zero-order. *arXiv preprint arXiv:2506.04430*, 2025.

- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21 (140):1–67, 2020.
- Shamshoum, Y., Hodos, N., Sieradzki, Y., and Schuster, A. CompAct: Compressed activations for memory-efficient LLM training. In Chiruzzo, L., Ritter, A., and Wang, L. (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 1511–1524, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.71.
- Shazeer, N. and Stern, M. Adafactor: Adaptive learning rates with sublinear memory cost. In Dy, J. and Krause, A. (eds.), *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 4596–4604. PMLR, 10–15 Jul 2018.
- Shu, Y., Zhang, Q., He, K., and Dai, Z. Refining adaptive zeroth-order optimization at ease. *arXiv preprint arXiv:2502.01014*, 2025.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., and Bowman, S. Glue: A multi-task benchmark and analysis platform for natural language understanding. In Linzen, T., Chrupała, G., and Alishahi, A. (eds.), *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pp. 353–355, Brussels, Belgium, November 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-5446.
- Yang, Y., Shi, Y., Wang, C., Zhen, X., Shi, Y., and Xu, J. Reducing fine-tuning memory overhead by approximate and memory-sharing backpropagation. *arXiv preprint arXiv:2406.16282*, 2024.
- Zhang, Y., Chen, C., Li, Z., Ding, T., Wu, C., Kingma, D. P., Ye, Y., Luo, Z.-Q., and Sun, R. Adam-mini: Use fewer learning rates to gain more. *arXiv preprint arXiv:2406.16793*, 2024a.
- Zhang, Y., Li, P., Hong, J., Li, J., Zhang, Y., Zheng, W., Chen, P.-Y., Lee, J. D., Yin, W., Hong, M., et al. Revisiting zeroth-order optimization for memory-efficient llm fine-tuning: A benchmark. *arXiv preprint arXiv:2402.11592*, 2024b.
- Zhang, Y., Liu, Y., and Fang, C. Adapm: a partial momentum algorithm for llm training. *arXiv preprint arXiv:2510.09103*, 2025.
- Zhao, J., Zhang, Z., Chen, B., Wang, Z., Anandkumar, A., and Tian, Y. Galore: Memory-efficient llm training by gradient low-rank projection. *arXiv preprint arXiv:2403.03507*, 2024a.
- Zhao, Y., Dang, S., Ye, H., Dai, G., Qian, Y., and Tsang, I. W. Second-order fine-tuning without pain for llms: A hessian informed zeroth-order optimizer. *arXiv preprint arXiv:2402.15173*, 2024b.
- Zhu, H., Zhang, Z., Cong, W., Liu, X., Park, S., Chandra, V., Long, B., Pan, D. Z., Wang, Z., and Lee, J. Apollo: Sgd-like memory, adamw-level performance. In Zaharia, M., Joshi, G., and Lin, Y. (eds.), *Proceedings of Machine Learning and Systems*, volume 7. MLSys, 2025.

A. Implementation of PRAC

We present the PRAC method for compressing activations in Algorithm 1, and introduces its application during forward and backward propagation in Algorithm 2.

Algorithm 2 Activation Storage and Recovery for Backpropagation

Require: Input X , parameters θ , forward function f , ranks (r_1, r_2) , step intervals (T_1, T_2) , current training step t

- 1: **Forward Propagation:**
 - 2: Compute output: $Y \leftarrow f(X; \theta)$
 - 3: Store compression components (Algorithm 1): $\{Q_1^{(t)}, Q_2^{(t)}, \mathcal{X}_1^{(t)}, \mathcal{X}_2^{(t)}\} \leftarrow \mathcal{C}(X, r_1, r_2, T_1, T_2, t)$
 - 4: **Return:** Y
 - 5:
 - 6: **Backward Propagation:**
 - 7: Receive gradient w.r.t output: $\nabla_Y \mathcal{L} \leftarrow \frac{\partial \mathcal{L}}{\partial Y}$
 - 8: Reconstruct input approximation from cache: $\tilde{X} \leftarrow \mathcal{X}_1^{(t)} Q_1^{(t)} + \mathcal{X}_2^{(t)} Q_2^{(t)}$
 - 9: Compute parameter and input gradients: $\nabla_X \mathcal{L}, \nabla_\theta \mathcal{L} \leftarrow \text{backward}(\nabla_Y \mathcal{L}, \tilde{X}, \theta)$
 - 10: **Return:** $\nabla_\theta \mathcal{L}, \nabla_X \mathcal{L}$
-

B. Memory Complexity Analysis

We take GPT structure as an example to show theoretical activation memory of PRAC. We assume a uniform rank ratio $r_1, r_2 \geq 0$ for the principal and random subspaces, respectively. Usually, $r_1 + r_2 \leq 0.6$.

Table 6. Comparison of activation stored in the GPT series architecture, with decreased activation values highlighted in red for PRAC.

Operation	Activation Saved	
	Baseline	PRAC ($0 \leq r_1 + r_2 \leq 0.6$)
$X = \text{LayerNorm}(\tilde{X})$	$\tilde{X} \in \mathbb{R}^{b \times s \times n}$ $(\mu(\tilde{X}), \sigma(\tilde{X})^2) \in \mathbb{R}^{b \times s}$	$\tilde{X} P_x \in \mathbb{R}^{b \times s \times (nr_1)}, \tilde{X} Q_x \in \mathbb{R}^{b \times s \times (nr_2)}$ $(\mu(X), \sigma(X)^2) \in \mathbb{R}^{b \times s}$
$q = X W_q$ $k = X W_k$ $v = X W_v$	$X \in \mathbb{R}^{b \times s \times n}$	$X P_q \in \mathbb{R}^{b \times s \times (nr_1)}$ $X Q_q \in \mathbb{R}^{b \times s \times (nr_2)}$
reshape q, k, v to $(b, h, s, n/h)$	None	None
$A_h = \text{flash_attn}(q, k, v)$	$q, k, v \in \mathbb{R}^{b \times h \times s \times n/h}$ two buffers $\in \mathbb{R}^{b \times s \times h}$	$q, k, v \in \mathbb{R}^{b \times h \times s \times n/h}$ two buffers $\in \mathbb{R}^{b \times s \times h}$
reshape A_h to (b, s, n)	None	None
$A''_o = A_h W_o^T$	$A_h \in \mathbb{R}^{b \times s \times n}$	$A_h P_h \in \mathbb{R}^{b \times s \times (nr_1)}$ $A_h Q_h \in \mathbb{R}^{b \times s \times (nr_2)}$
residual: $A'_o = A''_o + \tilde{X}$	None	None
$A_o = \text{LayerNorm}(A'_o)$	$A'_o \in \mathbb{R}^{b \times s \times n}$ $(\mu(A'_o), \sigma(A'_o)^2) \in \mathbb{R}^{b \times s}$	$A'_o P'_o \in \mathbb{R}^{b \times s \times (nr_1)}, A'_o Q'_o \in \mathbb{R}^{b \times s \times (nr_2)}$ $(\mu(A'_o), \sigma(A'_o)^2) \in \mathbb{R}^{b \times s}$
$A'_u = A_o W_{\text{up}}^T$	$A_o \in \mathbb{R}^{b \times s \times n}$	$A_o P_o \in \mathbb{R}^{b \times s \times (nr_1)}$ $A_o Q_o \in \mathbb{R}^{b \times s \times (nr_2)}$
$A_u = \text{GeLU}(A'_u)$	$A'_u \in \mathbb{R}^{b \times s \times m}$	$A'_u P'_u \in \mathbb{R}^{b \times s \times (mr_1)}$ $A'_u Q'_u \in \mathbb{R}^{b \times s \times (mr_2)}$
$A'_d = A_u W_{\text{down}}^T$	$A_u \in \mathbb{R}^{b \times s \times m}$	$A_u P_u \in \mathbb{R}^{b \times s \times (mr_1)}$ $A_u Q_u \in \mathbb{R}^{b \times s \times (mr_2)}$
residual: $A_d = A'_d + A'_o$	None	None

A comparison of activation compression between PRAC and baseline methods is presented in Table 6. The proposed PRAC compresses both linear layers (including the projection layers in Attention and the MLP) and non-linear layers (including LayerNorm and the GeLU activation). The mean and variance results in LayerNorm are not compressed, as they contribute negligibly to the total memory footprint (i.e., $bs \ll bsn$). Flash Attention operations are not compressed by PRAC due to their intricate coupled structure and engineering optimizations. Nevertheless, the compressions described above are sufficient for PRAC to achieve an overall memory reduction of nearly 40% in large-batch-size training tasks.

C. Theoretical Results for PRAC

C.1. Notations and Useful Lemma

Definition C.1 (Orthogonal Group). For a positive integer m , let $M_m(\mathbb{R})$ denote the set of all $m \times m$ real matrices. The orthogonal group, denoted $\mathcal{O}(m)$, is defined as:

$$\mathcal{O}(m) := \{A \in M_m(\mathbb{R}) \mid A^\top A = I_m\}. \quad (8)$$

Lemma C.2. Let $\tilde{S} \in \mathbb{R}^{m \times k}$ be a random matrix whose entries are independent and identically distributed (i.i.d.) standard normal variables, i.e., $\tilde{S}_{ij} \stackrel{i.i.d.}{\sim} \mathcal{N}(0, 1)$. Consider the (thin) QR decomposition of $\tilde{S} = VR$, where $V \in \mathbb{R}^{m \times k}$ satisfies $V^\top V = I_k$ and $R \in \mathbb{R}^{k \times k}$ is an upper-triangular matrix with positive diagonal entries. Then for any orthogonal matrix $O \in \mathcal{O}(m)$, OV and V are identically distributed, i.e.

$$OV \stackrel{d}{=} V \quad \forall O \in \mathcal{O}(m). \quad (9)$$

Proof. Let $O \in \mathcal{O}(m)$ be an arbitrary fixed orthogonal matrix and set $\tilde{S}' = O\tilde{S}$. Consider the QR decomposition of $\tilde{S}$ and $\tilde{S}'$, we have

$$\tilde{S} = VR, \tilde{S}' = V'R', \quad (10)$$

where $V, V' \in \mathbb{R}^{m \times k}$ satisfy $V^\top V = I_k, (V')^\top V' = I_k$ and R, R' are upper-triangular matrices with positive diagonal entries. Since the standard normal distribution is rotationally invariant, $\tilde{S}'$ has the same distribution as $\tilde{S}$, i.e., $\tilde{S}' \stackrel{d}{=} \tilde{S}$, therefore we have $V \stackrel{d}{=} V'$. What's more,

$$\tilde{S}' = O\tilde{S} = OVR. \quad (11)$$

Comparing (10) and (11), from the uniqueness of the thin QR decomposition we have $V' = OV, R' = R$, then

$$V' = OV \stackrel{d}{=} V \quad \forall O \in \mathcal{O}(m). \quad (12)$$

□

Lemma C.3. Suppose the matrix $M \in \mathbb{R}^{m \times m}$ satisfies $MO = OM \quad \forall O \in \mathcal{O}(m)$, then M must be a scalar matrix (i.e., $M = cI_m$).

Proof. We construct the reflection matrix as follows:

$$J_i = \text{diag}(1, \dots, 1, \underbrace{-1}_i, 1, \dots, 1) \in \mathbb{R}^{m \times m}, \quad (13)$$

where only the i -th diagonal entry is -1 and the others are 1. Clearly, $J_i \in \mathcal{O}(m)$. From the commutativity condition $J_i M = M J_i$, compare the entries in the i -th row and j -th column (with $i \neq j$), we have

$$(J_i M)_{ij} = -M_{ij}, (M J_i)_{ij} = M_{ij}. \quad (14)$$

Equating the two expressions gives:

$$-M_{ij} = M_{ij} \Rightarrow M_{ij} = 0 \quad (i \neq j).$$

Therefore, M is a diagonal matrix. Denote it as:

$$M = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_m). \quad (15)$$

Then consider a permutation matrix P_{ij} that swaps the i -th and j -th rows (and columns). It is also orthogonal, with $P_{ij}^{-1} = P_{ij}^\top = P_{ij}$. From the commutativity condition $P_{ij}M = MP_{ij}$, we have:

$$M = P_{ij}MP_{ij}. \quad (16)$$

Conjugating M by P_{ij} swaps the i -th and j -th diagonal entries of M . Since the matrices M and $P_{ij}MP_{ij}$ are equal, their diagonal entries must coincide, giving $\lambda_i = \lambda_j$ for any pair i, j . Hence, M must be a scalar matrix (i.e., $M = cI_m$). $\square$

Lemma C.4. Suppose the random matrix $V \in \mathbb{R}^{m \times k}$ satisfies $V^\top V = I_k$ and $\forall O \in \mathcal{O}(m)$, $OV \stackrel{d}{=} V$. Then we have

$$\mathbb{E}[VV^\top] = \frac{k}{m}I_m. \quad (17)$$

Proof. For any orthogonal matrix $O \in \mathcal{O}(m)$, OV has the same distribution as V . Then,

$$\mathbb{E}[VV^\top] = \mathbb{E}[(OV)(OV)^\top] = O\mathbb{E}[VV^\top]O^\top,$$

due to $O^\top = O^{-1}$,

$$\mathbb{E}[VV^\top]O = O\mathbb{E}[VV^\top] \quad \forall O \in \mathcal{O}(m).$$

From Lemma C.3, the expectation $\mathbb{E}[VV^\top]$ of the random matrix must be a scalar matrix, i.e.,

$$\mathbb{E}[VV^\top] = cI_m. \quad (18)$$

The constant c is determined by computing the trace:

$$\text{tr}(\mathbb{E}[VV^\top]) = \mathbb{E}[\text{tr}(VV^\top)] = \mathbb{E}[\text{tr}(V^\top V)] = k. \quad (19)$$

Meanwhile, $\text{tr}(cI_m) = cm$, thus $cm = k \Rightarrow c = \frac{k}{m}$. $\square$

Lemma C.5. Suppose $Q_1 \in \mathbb{R}^{n \times r_1}$ be a fixed orthogonal matrix, random matrix S_o be generated by (5), and let the random matrix $Q_2 \in \mathbb{R}^{n \times r_2}$ be the first component of $\text{QR}(S_o)$, then we have:

$$\mathbb{E}[Q_2Q_2^\top] = \frac{r_2}{n - r_1}(I_n - Q_1Q_1^\top). \quad (20)$$

Proof. Let $U \in \mathbb{R}^{n \times (n-r_1)}$ be an orthonormal basis for the orthogonal complement of P :

$$U^\top U = I_{n-r_1}, Q_1^\top U = O_{r_1 \times (n-r_1)}, Q_1Q_1^\top + UU^\top = I_n, \quad (21)$$

then

$$S_o = S - Q_1Q_1^\top S = UU^\top S. \quad (22)$$

We define $\tilde{S} = U^\top S \in \mathbb{R}^{(n-r_1) \times r_2}$, then $S_o = U\tilde{S}$. Since U has orthonormal columns and the entries of S are i.i.d Gaussian, the entries of $\tilde{S}$ are also i.i.d Gaussian. Perform QR decomposition on $\tilde{S}$: $\tilde{S} = VR$, then

$$S_o = U\tilde{S} = (UV)R, \quad (23)$$

according to the uniqueness of the thin QR decomposition, the projection matrix in the algorithm satisfies $Q_2 = UV \in \mathbb{R}^{n \times r_2}$. Then,

$$\begin{aligned} \mathbb{E}[Q_2Q_2^\top] &= \mathbb{E}[UVV^\top U^\top] = U\mathbb{E}[VV^\top]U^\top \\ &\stackrel{(a)}{=} U\left(\frac{r_2}{n - r_1}I_{n-r_1}\right)U^\top \\ &= \frac{r_2}{n - r_1}(I_n - Q_1Q_1^\top) \end{aligned} \quad (24)$$

where the equality (a) holds according to Lemma C.4 and $V \in \mathbb{R}^{(n-r_1) \times r_2}$. $\square$

C.2. Omitted Proofs in Section 4

Theorem C.6 (Unbiased Reconstruction of PRAC). *Let $P \in \mathbb{R}^{n \times (r_1 + r_2)}$ be the projection matrix defined in (6). If the scaling factor is set to $k = \frac{n-r_1}{r_2}$, the reconstruction $\tilde{X} = XPP^\top$ satisfies $\mathbb{E}[\tilde{X}] = X$.*

Proof. The expectations for the reconstruction $\tilde{X}$ are as follows:

$$\begin{aligned} \mathbb{E}[\tilde{X}] &= \mathbb{E}[XQ_1Q_1^\top + kXQ_2Q_2^\top] = XQ_1Q_1^\top + kX\mathbb{E}[Q_2Q_2^\top] \\ &\stackrel{(a)}{=} XQ_1Q_1^\top + k\frac{r_2}{n-r_1}X(I_n - Q_1Q_1^\top) \\ &= XQ_1Q_1^\top + \frac{n-r_1}{r_2}\frac{r_2}{n-r_1}X(I_n - Q_1Q_1^\top) \\ &= X, \end{aligned} \tag{25}$$

where in (a) we use Lemma C.5 to replace the $\mathbb{E}[Q_2Q_2^\top]$ term. $\square$

Theorem C.7 (Unbiased Reconstruction of RAC). *Let $Q_2 \in \mathbb{R}^{n \times r_2}$ sample uniformly from the Stiefel manifold. Set the scaling factor $k = \frac{n}{r_2}$. Then the reconstruction $\tilde{X}_{rac} = (kXQ_2)Q_2^\top$ satisfies $\mathbb{E}[\tilde{X}_{rac}] = X$.*

Proof. This result follows as a special case of Theorem C.6 by setting $r_1 = 0$ and taking $k = n/r_2$. $\square$

Theorem C.8 (Variance Bound of PRAC). *Under the conditions of Theorem C.6, the reconstruction error variance is bounded by:*

$$\mathbb{E}[\|\tilde{X} - X\|_F^2] \leq \left(\frac{n-r_1}{r_2} - 1\right) \|X - XQ_1Q_1^\top\|_F^2.$$

Proof. We evaluate the expected squared Frobenius norm of the reconstruction error,

$$\mathbb{E}[\|X - \tilde{X}\|_F^2] = \mathbb{E}[\|X(I_n - Q_1Q_1^\top - kQ_2Q_2^\top)\|_F^2]. \tag{26}$$

Recall from the properties of Q_1 that $I_n - Q_1Q_1^\top = UU^\top$. Substituting this relation, we obtain

$$\mathbb{E}[\|X - \tilde{X}\|_F^2] = \mathbb{E}[\|X(UU^\top - kQ_2Q_2^\top)\|_F^2] = \mathbb{E}[\text{tr}(X(UU^\top - kQ_2Q_2^\top)^2X^\top)], \tag{27}$$

where the last equality follows from the identity $\|A\|_F^2 = \text{tr}(AA^\top)$. Then, we compute the expectation of the squared matrix term:

$$\mathbb{E}[(UU^\top - kQ_2Q_2^\top)^2] = UU^\top - 2k\mathbb{E}[Q_2Q_2^\top] + k^2\mathbb{E}[Q_2Q_2^\top] \tag{28}$$

From Lemma C.4, we have $\mathbb{E}[Q_2Q_2^\top] = \frac{1}{k}UU^\top$. Substituting this yields

$$\mathbb{E}[(UU^\top - kQ_2Q_2^\top)^2] = UU^\top(1 - 2k\frac{1}{k} + k^2\frac{1}{k}) = UU^\top(k - 1), \tag{29}$$

Substituting (29) into (27), we have:

$$\mathbb{E}[\|X - \tilde{X}\|_F^2] = \text{tr}(XUU^\top X^\top)(k - 1) = \left(\frac{n-r_1}{r_2} - 1\right) \|X - XQ_1Q_1^\top\|_F^2 \tag{30}$$

$\square$

Theorem C.9 (Variance Bound of RAC). *Under the conditions of Theorem C.7, then the reconstruction error variance is bounded by:*

$$\mathbb{E}[\|\tilde{X}_{rac} - X\|_F^2] \leq \left(\frac{n}{r_2} - 1\right) \|X\|_F^2.$$

Proof. This result follows as a special case of Theorem C.8 by setting $r_1 = 0$. $\square$

Theorem C.10 (Lower Bound). *Under Assumption 4.1, if $s < r < n$, then*

$$(4) \geq \left(\frac{n-s}{r-s} - 1 \right) q.$$

Proof. Without loss of generality, assume the activation matrix is $X = \text{diag}(\sigma_1, \dots, \sigma_n) \in \mathbb{R}^{n \times n}$ with $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n > 0$ and, by Assumption 4.1, $\sum_{i=s+1}^n \sigma_i^2 \leq q$. The unbiasedness condition $\mathbb{E}[XPP^\top] = X$ implies $\mathbb{E}[PP^\top] = I$. Set $A = PP^\top \in \mathbb{R}^{n \times n}$; then the expected error can be written as

$$\epsilon = \mathbb{E} \|X(A - I)\|_F^2 = \sum_{i=1}^n \sigma_i^2 \left[\mathbb{E}(A_{ii} - 1)^2 + \sum_{j \neq i} \mathbb{E} A_{ij}^2 \right] \equiv \sum_{i=1}^n \sigma_i^2 \alpha_i, \quad (31)$$

where $\alpha_i \geq 0$ is defined by the bracket.

1. **Vanishing of the first s coefficients.** Consider a family of X where $\sigma_i = t$ for $i \leq s$ while $\sigma_{s+1}, \dots, \sigma_n$ are fixed and satisfy the sum-of-squares constraint. If for some distribution of P we have $\sum_{i=1}^s \alpha_i > 0$, then sending $t \rightarrow \infty$ would make ϵ arbitrarily large, so $\sup_X \inf_P \epsilon = \infty$ and the bound holds trivially. Hence we need only consider strategies with $\sum_{i=1}^s \alpha_i = 0$, i.e. $\alpha_i = 0$ for all $i \leq s$. This forces P to be block-diagonal almost surely:

$$P = \begin{bmatrix} I_s & 0 \\ 0 & \tilde{P} \end{bmatrix}, \quad \tilde{P} \in \mathbb{R}^{(n-s) \times (r-s)}. \quad (32)$$

Consequently the error reduces to

$$\epsilon = \sum_{i=s+1}^n \alpha_i \sigma_i^2. \quad (33)$$

2. **Lower bound on the sum of the remaining α_i .** Let $\tilde{A} = \tilde{P}\tilde{P}^\top \in \mathbb{R}^{(n-s) \times (n-s)}$. From $\mathbb{E}[A] = I$ we obtain $\mathbb{E}[\text{tr}(\tilde{A})] = \mathbb{E}[\text{tr}(A)] - s = n - s$. Because $\text{rank}(\tilde{A}) \leq r - s$, Cauchy–Schwarz gives

$$\text{tr}(\tilde{A}^2) \geq \frac{[\text{tr}(\tilde{A})]^2}{r - s}. \quad (34)$$

Taking expectations and using Jensen’s inequality,

$$\mathbb{E}[\text{tr}(\tilde{A}^2)] \geq \frac{(\mathbb{E}[\text{tr}(\tilde{A})])^2}{r - s} = \frac{(n - s)^2}{r - s}. \quad (35)$$

Now compute the sum of α_i for $i > s$:

$$\begin{aligned} \sum_{i=s+1}^n \alpha_i &= \sum_{i=s+1}^n \left[\mathbb{E}(A_{ii} - 1)^2 + \sum_{j \neq i} \mathbb{E} A_{ij}^2 \right] \\ &= \sum_{i=s+1}^n \left[\sum_{j=1}^n \mathbb{E} A_{ij}^2 - 1 \right] \quad (\text{since } \mathbb{E} A_{ii} = 1) \\ &= \sum_{i=s+1}^n \sum_{j=s+1}^n \mathbb{E} A_{ij}^2 - (n - s) \quad (\text{by the block-diagonal form}) \\ &= \mathbb{E}[\text{tr}(\tilde{A}^2)] - (n - s) \\ &\geq \frac{(n - s)^2}{r - s} - (n - s) \equiv C. \end{aligned} \quad (36)$$

3. **Completion of the lower bound.** For any fixed $\sigma_{s+1}, \dots, \sigma_n$, using $\alpha_i \geq 0$ we have

$$\sum_{i=s+1}^n \alpha_i \sigma_i^2 \geq \left(\sum_{i=s+1}^n \alpha_i \right) \min_{s+1 \leq i \leq n} \sigma_i^2 \geq C \cdot \min_i \sigma_i^2. \quad (37)$$

Taking the infimum over admissible α_i (i.e. over distributions of P) and then the supremum over σ_i satisfying $\sum_{i=s+1}^n \sigma_i^2 \leq q$ yields

$$\sup_{\sigma_i} \inf_{\alpha_i} \sum_{i=s+1}^n \alpha_i \sigma_i^2 \geq C \cdot \sup_{\sigma_i} \min_i \sigma_i^2. \quad (38)$$

Under the sum-of-squares constraint, $\sup \min_i \sigma_i^2$ is attained when all σ_i^2 are equal, i.e. $\sigma_i^2 = q/(n-s)$; hence $\sup \min_i \sigma_i^2 = q/(n-s)$. Substituting this together with the value of C from (36) gives

$$\sup_{\sigma_i} \inf_{\alpha_i} \sum_{i=s+1}^n \alpha_i \sigma_i^2 \geq \left(\frac{(n-s)^2}{r-s} - (n-s) \right) \frac{q}{n-s} = \left(\frac{n-s}{r-s} - 1 \right) q. \quad (39)$$

By (33) this is exactly a lower bound for the quantity (4), completing the proof. $\square$

D. Experiment Details

D.1. Pre-training Setting

We detail the architectural configurations and pre-training hyperparameters for both the LLaMA and GPT models. To ensure numerical stability and computational efficiency, all LLaMA models are trained using bfloat16 precision and GPT models are trained using Automatic Mixed Precision. The key hyperparameters for LLaMA and GPT models across different scales are summarized in Table 7.

LLaMA Settings. The LLaMA models are trained with a maximum sequence length of 256 and a global batch size of 512. We employ a learning rate schedule comprising a linear warmup over the first 10% of training steps, followed by a cosine annealing phase that decays the learning rate to 10% of its peak value. Across all LLaMA model scales, we adopt the optimal learning rates reported in the original papers for comparison methods. For PRAC, the learning rate is selected from a grid ranging from 0.5×10^{-4} to 4×10^{-3} in steps of 0.5×10^{-4} , which is closely aligned with the settings used by the baselines.

GPT Settings. For the GPT models, we adopt a maximum sequence length of 1024 while maintaining a global batch size of 512. We set the warmup steps to 2K, which corresponds to 4% of the total steps³. Notably, given that the official training code and hyperparameters for GaLore (Zhao et al., 2024a) and RSO (Chen et al., 2025) on GPT architectures are not publicly available, we train these models using learning rates aligned with the baseline. For these methods, we configure the ranks as 256 and 512 for the 124M and 355M models, respectively.

Table 7. Hyperparameter settings for pre-training LLaMA and GPT-2 model using the PRAC method.

	Params	Hidden	Intermediate	Heads	Layers	Steps
LLaMA	35M	384	1024	8	6	5K
	60M	512	1376	8	8	10K
	130M	768	2048	12	12	20K
	350M	1024	2736	16	24	60K
	1B	2048	5461	24	32	150K
GPT-2	124M	768	3072	12	12	50K
	355M	1024	4096	16	24	50K

D.2. Fine-tuning Setting

We fine-tune the pre-trained RoBERTa-Base model on the GLUE benchmark using the Hugging Face library⁴. All tasks are trained for 30 epochs with a batch size of 16. Refer to Table 8 for the specific hyperparameters employed for PRAC.

³<https://github.com/zyushun/Adam-mini/tree/main>

⁴https://huggingface.co/transformers/model_doc/roberta.html

Table 8. Hyperparameter settings for fine-tuning RoBERTa-Base model on the GLUE benchmark using the PRAC method.

	COLA	STSB	MRPC	RTE	SST2	MNLI	QNLI	QQP
Batch Size	16	16	16	16	16	16	16	16
Epochs	30	30	30	30	30	30	30	30
Learning Rate	3E-05	3E-05	3E-05	3E-05	1E-05	1E-05	1E-05	1E-05
Rank ($r_1 + r_2$)	4							
PRAC Interval	200							
Max Seq Length	512							

D.3. Additional Result

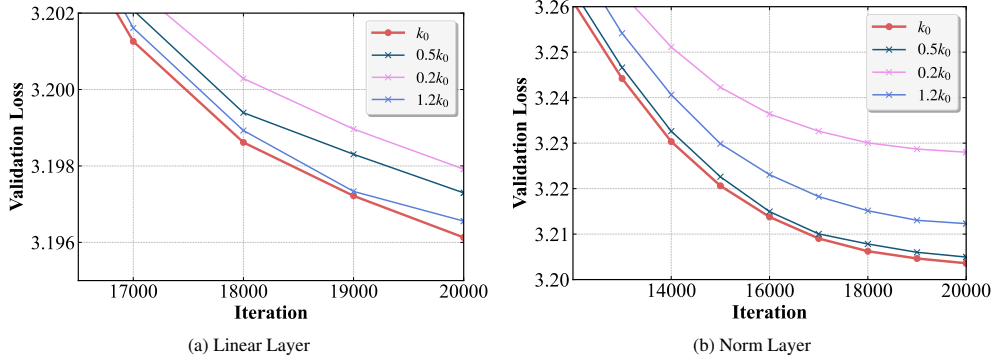
Hyperparameter Sensitivity. Specifically, we fix the total compression rank to $r_1 + r_2 = \lfloor 0.6n \rfloor$ for linear layers of LLaMA-130M. We then vary the principal rank ratio $r_1/(r_1 + r_2)$, and correspondingly adjust the scaling factor $k = (n - r_1)/r_2$ to maintain unbiasedness as prescribed by Theorem 4.3.

Table 9 reports the final performance after 20K training steps. This empirical finding validates our theoretical framework: the principal subspace captures dominant information to minimize variance, while the random subspace ensures unbiasedness. The sweet spot $r_1/(r_1 + r_2) = 0.5$ (i.e., $r_1 = r_2$) achieves the optimal trade-off.

 Table 9. Final validation loss of LLaMA-130M under different principal-to-random rank allocations (fixed total rank $r_1 + r_2 = \lfloor 0.6n \rfloor$).

$r_1/(r_1 + r_2)$	0/6 (RAC)	1/6	2/6	3/6	4/6	5/6	6/6 (PAC)
Validation Loss	3.239	3.209	3.211	3.207	3.212	3.216	3.229

Selection of the Scaling Factor k . We evaluate different values of scaling factor k on the linear and normalization layers of LLaMA-130M. Defining the theoretical baseline as $k_0 = \frac{n-r_1}{r_2}$, we test $k \in \{k_0, 0.5k_0, 0.2k_0, 1.2k_0\}$. The results in Figure 8 indicate that $k = k_0$ yields the best convergence, whereas other settings result in performance degradation. This empirical evidence aligns with our theoretical findings (See Section 4.3).


 Figure 8. Loss curves of using different k in LLaMA-130M pre-training, where $k_0 = \frac{n-r_1}{r_2}$. Setting $k = k_0$ performs better than larger or smaller settings, whether in linear or nonlinear layers.

Compatibility with Advanced Optimizers. PRAC is orthogonal to optimizer choice. We integrate PRAC with memory-efficient optimizers Muon (Liu et al., 2025) and Adam-mini (Zhang et al., 2024a). As shown in Figure 9 and Table 10, PRAC successfully reduces memory by an additional 30% on top of these optimizers with negligible perplexity degradation, highlighting its versatility.

Comparison with Engineering Heuristics Gradient Checkpointing (GC) recomputes activations from checkpoints during backpropagation, often requiring multiple forward passes, which incurs approximately 30% extra computation (see Table 11). In contrast, the time overhead of PRAC consists of a relatively small amount of low-dimensional computation, as analyzed in Section 5.3.

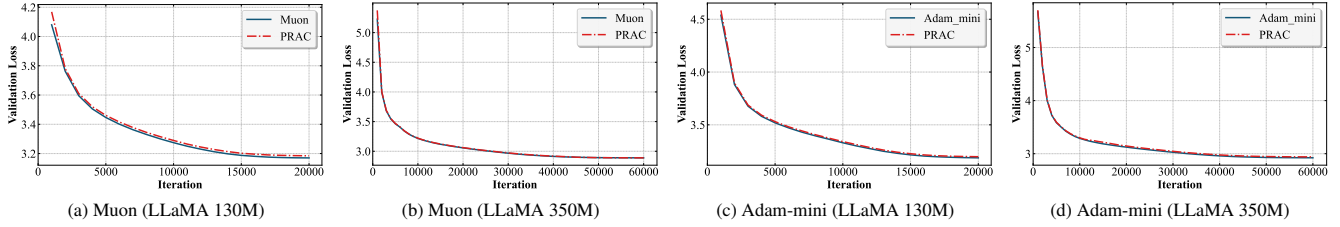


Figure 9. Loss curves of pre-training LLaMA model based on the Muon and Adam-mini optimizers w/o. PRAC.

Table 10. Comparison with memory-efficient algorithms on pretraining LLaMA. Report validation perplexity (PPL, lower is better) and the algorithm’s peak memory usage (lower is better).

Method	PRAC	LLaMA-130M		LLaMA-350M	
		Ppl	Mem	Ppl	Mem
Muon	×	23.90	21.04	17.98	45.26
	✓	24.16	15.37 (↓ 27%)	18.04	30.82 (↓ 32%)
Adam-mini	×	24.22	21.01	18.60	45.42
	✓	24.48	15.36 (↓ 27%)	18.92	31.09 (↓ 31%)

However, in practical implementation, PRAC compresses only linear layers and some non-linear layers, while certain layers (e.g., Flash Attention) remain uncompressed. Furthermore, GC uniformly skips these layers by saving inter-layer checkpoints. Therefore, for a fair comparison, we set up the following experiment: we match memory usage by tuning GC’s checkpointing layers and its batch size, and track validation loss over time. On LLaMA 130M, PRAC reaches each target validation loss faster than GC (see Table 12).

Table 11. Comparison of per-step time and throughput on LLaMA-130M (float32) under different batch sizes. PRAC compresses linear and RMSNorm layers, while GC recomputes activations. Throughput is measured in examples per second.

	Batch=32		Batch=64		Batch=128	
	Time (s)	Throughput	Time	Throughput	Time	Throughput
Baseline	0.289	443	0.585	437	1.211	423
PRAC	0.328 (↑ 13%)	389	0.663 (↑ 13%)	385	1.394 (↑ 15%)	367
GC	0.374 (↑ 29%)	342	0.761 (↑ 30%)	336	1.574 (↑ 30%)	325

Table 12. Time (in minutes) to reach target validation loss on LLaMA-130M. PRAC and GC are configured to use similar memory.

Method	Memory (GB)	Time (min) to reach target validation loss				
		4.0	3.8	3.6	3.4	3.2
PRAC	31.86	13.1	22.2	33.0	63.3	256.1
GC	31.59	15.2	25.5	38.2	72.8	294.5

Gradient accumulation (GA) fundamentally relies on serializing the computation. GA serializes computation by performing multiple sequential forward-backward passes before each parameter update, introducing significant overhead compared to a single large-batch update. In contrast, PRAC compresses activations within a single forward-backward pass, avoiding the repetitive kernel launches and synchronization overhead of GA, thus achieving higher throughput.

Beyond throughput, GA cannot reduce activation memory for a single micro-batch—it assumes a batch size of 1 fits in GPU memory. However, with long sequences (e.g., 4k–8k tokens), even batch size 1 can cause OOM due to large activations. PRAC directly compresses activations, reducing per-sample memory and enabling training on hardware where batch size 1 alone would fail—a capability GA lacks.